

Tunnijuhend: söökla arvustused

11IT · Veebirakendused · viies tund · tunnis

Täna teed oma esimese päris Reacti rakenduse: söökla toitute arvustused. Iga samm on antud koos koodiga ja iga koodirida on tabelis lahti seletatud. Teeme ainult **frontendi**: arvustused on failis, mis teeb näo, et on backend.

Kuidas töötada. Ära kopeeri koodi PDF-ist, vaid **kirjuta see ise ümber** ja loe seletus kõrvalt läbi. Nii märkad, mis mida teeb, ja kirjavigadest õpid rohkem kui kopeerimisest. Pärast igit sammu salvesta (Ctrl+S) ja vaata brauserit. Kui leht läheb tühjaks, vaata brauseri konsooli (F12): punane tekst ütleb, mis valesti on.

Samm 0: ettevalmistus

Ava eelmisel korral tehtud projekt VS Code'is, ava terminal (Terminal → New Terminal) ja käivita server. Ava terminali antud aadress brauseris.

```
npm run dev
```

Tühjenda fail `src/index.css`: vali kõik (Ctrl+A) ja kustuta. Laadi kursuse lehelt alla `App.css` ja pane see kausta `src` vana `App.css` asemele. Selles on kujundus valmis. Siis kustuta kogu `src/App.tsx` sisu ja kirjuta asemele:

`src/App.tsx`

```
import './App.css';

export default function App() {
  return (
    <main>
      <h1>Söökla arvustused</h1>
    </main>
  );
}
```

Kood	Mida see teeb
<code>import './App.css';</code>	Ühendab kujundusfaili. Kõik klassid, mida allpool kasutame, on seal.
<code>export default function App() {</code>	Reactis on iga lehe tükk komponent . Komponent on lihtsalt funktsioon, mille nimi algab suure tähega. <code>export default</code> lubab teistel failidel seda kasutada: <code>main.tsx</code> paneb just selle komponendi ekraanile.
<code>return (</code>	Mida funktsioon tagastab, seda React näitab ekraanil.
<code><main></code> <code><h1>Söökla arvustused</h1></code> <code></main></code>	See on JSX : näeb välja nagu HTML, aga elab JavaScripti sees. Peaaegu kõik on nagu HTML-is. Üks erinevus: CSS-klassi jaoks kirjutatakse <code>className</code> , mitte <code>class</code> .

Brauseris peaks nüüd olema ainult pealkiri heledal taustal.

Samm 1: mis kujul on üks arvustus

Enne kui midagi ekraanile paneme, lepime kokku, mis on üks arvustus. Konspektis nimetasime seda frontendi ja backendi kokkuleppeks. Tee kausta `src` uus fail `types.ts` (paremklõps kaustal → New File):

src/types.ts

```
export type Arvustus = {  
  id: number;  
  toit: string;  
  hinne: number;  
  kommentaar: string;  
};
```

Kood	Mida see teeb
export type Arvustus = {	Teeme uue tüübi nimega Arvustus. Tüüp ei tee ise midagi, ta kirjeldab, mis kujul andmed on. export: teised failid saavad seda kasutada.
id: number;	Igal arvustusel on oma number, et neid eristada. Päril rakenduses paneb selle andmebaas.
toit: string;	Toidu nimi. string tähendab teksti.
hinne: number;	Hinne arvuna, 1 kuni 5.
kommentaar: string;	Paar sõna toidu kohta.

Nüüd teeme nädisandmed, mis teevad näo, et tulid backendist. Uus fail src/andmed.ts:

src/andmed.ts

```
import type { Arvustus } from './types';  
  
export const naidisArvustused: Arvustus[] = [  
  { id: 1, toit: 'Kanakaste riisiga', hinne: 5, kommentaar: 'Parim toit sel nädalal!' },  
  { id: 2, toit: 'Hernesupp', hinne: 3, kommentaar: 'Natuke liiga paks.' },  
  { id: 3, toit: 'Makaronid hakklihaga', hinne: 4, kommentaar: 'Klassika, alati hea.' },  
];
```

Kood	Mida see teeb
import type { Arvustus } from './types';	Toome tüübi failist types.ts. Sõna type on vajalik, sest impordime ainult tüüpi. Ilma selleta annab see projekt vea.
export const naidisArvustused: Arvustus[] = [{ id: 1, toit: 'Kanakaste riisiga', ... },	Massiiv nimega naidisArvustused. : Arvustus[] ütleb TypeScriptile: see on massiiv ja iga element peab olema Arvustus.
	Üks arvustus on objekt: loogelistes sulgudes väljad ja nende väärtused. Objektid on komaga eraldatud.

Proovi: muuda esimeses arvustuses hinne: 5 kujule hinne: "viis". VS Code joonib selle punaseks. Vii hiir punase joone peale ja loe, mida TypeScript ütleb. See ongi tüüpide mõte: viga leitakse enne, kui keegi lehte avab. Muuda tagasi.

Samm 2: näita arvustusi lehel

Paneme arvustused ekraanile. Kirjuta src/App.tsx üle:

src/App.tsx

```
import { naidisArvustused } from './andmed';
import './App.css';

export default function App() {
  return (
    <main>
      <h1>Söökla arvustused</h1>
      <ul>
        {naidisArvustused.map((a) => (
          <li key={a.id} className="kaart">
            <strong>{a.toit}</strong> ({a.hinne}/5)
            <p>{a.kommentaar}</p>
          </li>
        ))}
      </ul>
    </main>
  );
}
```

Kood	Mida see teeb
import { naidisArvustused } from './andmed';	Toome näidisandmed. Siin ei ole sõna type, sest see on päris väärtus, mitte tüüp.
	Tavaline HTML-i nimekiri.
{naidisArvustused.map((a) => (	Loogelised sulud JSX-is tähendavad: siin on JavaScript. <code>.map()</code> käib massiivi läbi ja teeb igast elemendist midagi uut. <code>a</code> on parasjagu käes olev arvustus. Iga arvustuse kohta tagastame ühe <code></code> -i.
<li key={a.id} className="kaart">	<code>key</code> on Reacti jaoks: iga nimekirja rida vajab unikaalset võtit, et React teaks, milline rida on milline. Sobib <code>id</code> . <code>className</code> annab kujunduse.
{a.toit} ({a.hinne}/5)	<code>{a.toit}</code> paneb sinna arvustuse toidu nime. Kõik loogelistes sulgudes arvutatakse ja näidatakse.
{a.kommentaar}	Kommentaar oma lõigus.
))}	Sulgeb <code>map</code> -i: ümarsulg, ümarsulg, loogeline sulg.

Proovi: lisa `andmed.ts`-i neljas arvustus oma lemmiktoiduga. Leht uueneb ise ja uus arvustus on kohe nimekirjas. Pane uuele arvustusele `id`, mida teistel pole!

Samm 3: oma komponent

App läheb pikaks. Reactis tõstetakse korduv tükk oma komponenti, mida saab siis kasutada nagu uut HTML-i silti. Teeme komponendi ühe arvustuse kaardi jaoks. Uus fail `src/ArvustuseKaart.tsx` (lõpp on `.tsx`, sest selles on JSX):

src/ArvustuseKaart.tsx

```
import type { Arvustus } from './types';

type Props = {
  arvustus: Arvustus;
};

export default function ArvustuseKaart({ arvustus }: Props) {
  const tahed = '★'.repeat(arvustus.hinne) + '☆'.repeat(5 - arvustus.hinne);
  return (
    <li className="kaart">
      <div className="kaart-pea">
        <strong>{arvustus.toit}</strong>
        <span className="tahed">{tahed}</span>
      </div>
      <p>{arvustus.kommentaar}</p>
    </li>
  );
}
```

Kood	Mida see teeb
<pre>type Props = { arvustus: Arvustus; };</pre>	Propsid on andmed, mida komponent väljast saab, nagu funktsiooni argumentid või HTML-i atribuudid. Siin ütleme: see komponent saab ühe arvustuse.
<pre>function ArvustuseKaart({ arvustus }: Props)</pre>	Loogelised sulud võtavad propsidest välja arvustus-e, et saaksime seda otse kasutada.
<pre>const tahed = '★'.repeat(arvustus.hinne) + '☆'.repeat(5 - arvustus.hinne);</pre>	repeat kordab teksti: '★'.repeat(4) on ★★☆☆. Ülejäänud kohad täidame tühjade tähtedega, et kokku oleks alati viis. Tähemärgid saad siit kopeerida.
<pre><div className="kaart-pea"></pre>	Nimi ja tähed ühel real (vaata App.css-i).
<pre>{tahed}</pre>	Näitab ülal tehtud tähti.

Nüüd kasuta kaarti App-is. Muuda src/App.tsx:

src/App.tsx

```
import { naidisArvustused } from './andmed';
import ArvustuseKaart from './ArvustuseKaart';
import './App.css';

export default function App() {
  return (
    <main>
      <h1>Söökla arvustused</h1>
      <ul>
        {naidisArvustused.map((a) => (
          <ArvustuseKaart key={a.id} arvustus={a} />
        ))}
      </ul>
    </main>
  );
}
```

Kood	Mida see teeb
<pre>import ArvustuseKaart from './ArvustuseKaart';</pre>	Toome komponendi. Loogelisi sulge pole, sest kaart on export default.

Kood	Mida see teeb
<code><ArvustuseKaart key={a.id} arvustus={a} /></code>	Kasutame oma komponenti nagu HTML-i silti. <code>arvustus={a}</code> annab propsi. Kui kirjutad <code>arvustu={a}</code> , annab TypeScript kohe vea, sest sellist propsi pole.

Leht peaks nüüd olema ilusam: valged kaardid ja kollased tähed.

Samm 4: uue arvustuse lisamine

Seni oli lehel ainult see, mis failis kirjas. Nüüd tahame, et kasutaja saaks ise arvustuse lisada. Selleks on vaja Reacti **mälu**: `useState`. Kirjuta `App.tsx` viimast korda üle. See on pikk, kirjuta rahulikult:

`src/App.tsx`

```
import { useState } from 'react';
import type { FormEvent } from 'react';
import type { Arvustus } from './types';
import { naidisArvustused } from './andmed';
import ArvustuseKaart from './ArvustuseKaart';
import './App.css';

export default function App() {
  const [arvustused, setArvustused] = useState<Arvustus[]>(naidisArvustused);
  const [toit, setToit] = useState('');
  const [hinne, setHinne] = useState(5);
  const [kommentaar, setKommentaar] = useState('');

  function lisa(e: FormEvent<HTMLFormElement>) {
    e.preventDefault();
    const uus: Arvustus = { id: Date.now(), toit, hinne, kommentaar };
    setArvustused([uus, ...arvustused]);
    setToit('');
    setHinne(5);
    setKommentaar('');
  }

  return (
    <main>
      <h1>Söökla arvustused</h1>

      <form onSubmit={lisa}>
        <input value={toit} onChange={(e) => setToit(e.target.value)} placeholder="Mis toit?" />
        <select value={hinne} onChange={(e) => setHinne(Number(e.target.value))}>
          <option value={5}>5 ★</option>
          <option value={4}>4 ★</option>
          <option value={3}>3 ★</option>
          <option value={2}>2 ★</option>
          <option value={1}>1 ★</option>
        </select>
        <input
          value={kommentaar}
          onChange={(e) => setKommentaar(e.target.value)}
          placeholder="Kommentaar"
        />
        <button type="submit">Lisa</button>
      </form>
    </main>
  );
}
```

```

    <ul>
      {arvustused.map((a) => (
        <ArvustuseKaart key={a.id} arvustus={a} />
      ))}
    </ul>
  </main>
);
}

```

Kood	Mida see teeb
import { useState } from 'react';	useState on Reacti mälu. Kui seal olev väärtus muutub, joonistab React lehe uuesti.
import type { FormEvent } from 'react';	Vormi saatmise sündmuse tüüp, vaja allpool.
const [arvustused, setArvustused] = useState<Arvustus[]>(naidisArvustused);	Loob mälu arvustuste jaoks. Saame kaks asja: väärtuse arvustused ja funktsiooni setArvustused, millega seda muuta. Alguses on seal näidisandmed. Tavalise muutuja muutmisel leht ei uueneks, sellepärast on vaja state'i.
const [toit, setToit] = useState('');	Mälu selle jaoks, mida kasutaja praegu toidu kasti kirjutab. Alguses tühi tekst. Kommentaari jaoks on samasugune rida.
const [hinne, setHinne] = useState(5);	Validud hinne, alguses 5.
function lisa(e: FormEvent<HTMLFormElement>) {	See funktsioon käivitub, kui vorm saadetakse.
e.preventDefault();	Muidu teeks brauser vormi saatmisel lehele värskenduse ja kõik kaoks.
const uus: Arvustus = { id: Date.now(), toit, hinne, kommentaar };	Teeme uue arvustuse. Date.now() on millisekundid alates 1970. aastast, iga kord erinev, seega sobib id-ks. toit on lühend toit: toit-ist.
setArvustused([uus, ...arvustused]);	Teeme uue massiivi: ees on uus arvustus ja ... laotab vanad selle järele. Ära kasuta push-i: siis on massiiv sama ja React ei märka muutust.
setToit(''); setHinne(5); setKommentaari('');	Tühjendab vormi järgmise arvustuse jaoks.
<form onSubmit={lisa}>	Kui vajutad nuppu või Enterit, kutsub vorm välja meie funktsiooni lisa.
<input value={toit} onChange={(e) => setToit(e.target.value)} />	Kast näitab alati seda, mis on mälus (value). Iga klahvivajutuse peale (onChange) paneme uue teksti mällu. e.target.value on see, mis kastis praegu kirjas. Kommentaari kast töötab täpselt samamoodi.
<select value={hinne} onChange={(e) => setHinne(Number(e.target.value))>	Sama muster valikmenüüga. Valikmenüü annab alati teksti, näiteks "4". Number() teeb sellest arvu 4, sest hinne peab olema arv.
<option value={5}>5 ★</option>	Üks valik menüüs.
<button type="submit">Lisa</button>	Saadab vormi.
{arvustused.map((a) => (	Nimekiri tuleb nüüd mälust, mitte otse näidisandmetest. Kui mälu muutub, muutub ka nimekiri.

Proovi: kirjuta setHinne(Number(e.target.value)) asemele setHinne(e.target.value). Mida TypeScript ütleb? Miks? Muuda tagasi.

Viimane katse

Lisa paar arvustust ja vajuta siis **F5**. Kus su arvustused on? Mõtle konspekti peale: mida oleks vaja, et need alles jääksid ja et pinginaaber näeks sinu arvustusi oma arvutis? Sellest räägime järgmises tunnis.

Kui jõuad varem valmis

Need on väljakutsed: koodi siin pole, ainult vihjed. Proovi ise.

- **Ära luba tühja nime.** Kui toidu nimi on tühi, ei tohi midagi lisanduda. Vihje: funktsiooni `lis`a algusesse `if` ja `return`.
- **Kustuta-nupp.** Igal kaardil on nupp, mis eemaldab selle arvustuse. Vihje: `state` elab App-is, nii et funktsioon `kustuta(id)` peab ka olema App-is ja kaart saab selle propsina. Eemaldamiseks sobib `.filter()`.
- **Kokkuvõte.** Näita nimekirja kohal, mitu arvustust on ja mis on keskmine hinne. Vihje: `.length` ja `for`-tsükel.

Hoia projekt alles! Kopeeri kaust tunni lõpus mälupulgale või oma pilve. Järgmises tunnis teeme sellele backendi.